

Introductory Code Review

Case Study: Equipment Loan Fee

Format: Individual coding with paired peer review

Total: 10 marks

1. Scenario and Learning Objectives

A university laboratory charges a daily fee for borrowing equipment. A developer has written a short JavaScript function to calculate the total fee. Before the function is accepted, it must be reviewed and improved.

This coursework develops your ability to identify code-quality concerns, make a small justified improvement, check expected behaviour, and provide constructive peer feedback. Complete the programming work in **one JavaScript file**. No database, external packages, or automated testing framework is required.

2. Starter Code

In the following function, **d** represents the number of borrowing days and **r** represents the daily rate.

```
function fee(d, r) {  
  let total = d * r;  
  console.log(total);  
  return total;  
}
```

3. Task 1: Review the Original Code

Identify **three concerns** and briefly explain why each matters. Consider naming, input validation, and unnecessary console output. For each concern, propose an improvement.

Recognise that the multiplication already produces the expected total for valid ordinary inputs. A review should distinguish existing strengths from issues that require attention.

4. Task 2: Improve the Function

Revise the function to meet the following requirements:

1. Use a descriptive function name and meaningful parameter names.
2. Accept borrowing days only when supplied as a positive integer number.
3. Accept the daily rate only when supplied as a finite, nonnegative number.
4. Reject invalid inputs by throwing an **Error** with a clear message. Do not convert numeric strings into numbers.
5. Return the total fee for valid inputs without printing from inside the calculation function.

Keep the implementation short and readable. Update your example calls if you rename the function. For this introductory exercise, assume valid inputs are within an ordinary range for fee calculations.

In a short explanation, distinguish **refactoring** from **behaviour changes**. Renaming can preserve behaviour when callers are updated; adding validation and removing console output change observable behaviour.

5. Task 3: Check the Behaviour

Demonstrate the following five cases and record the actual result beside the expected result. You may use simple calls with `try...catch` in the same JavaScript file; a test framework is not required. Console output in demonstration code is permitted.

Days	Daily rate	Expected outcome
3	10	Returns 30.
1	0	Returns 0.
0	10	Throws an error.
2	-5	Throws an error.
"3"	10	Throws an error because the days value is a string.

Execute all five checks independently so that an expected error does not prevent later cases from running. The five checks are the required demonstrations; your implementation must still satisfy all requirements in Task 2.

6. Task 4: Participate in Peer Review

Work with one classmate. Each student must act as both an author and a reviewer.

1. Create a branch in your assigned repository and commit your improved code.
2. Open a pull request explaining what you changed, why, and how you checked it.
3. Ask your classmate to review the changes against the requirements.
4. As reviewer, leave at least one meaningful comment or question. Identify a specific observation and explain its relevance; do not invent a defect.
5. As author, respond to the feedback and revise your code where necessary. If no revision is needed, explain your reasoning.

For example, a reviewer might ask:

Does the validation reject a numeric string such as "3"? Please explain which check prevents it from being accepted.

7. Submission Requirements

Submit:

- One JavaScript file containing the improved function and demonstration calls.
- A brief note identifying the three concerns and explaining your changes.
- Evidence of the five checks, showing expected and actual results.
- Your pull-request link and evidence of the review you gave and received.

Use the deadline and submission channel announced by the instructor.

8. Assessment Criteria

Criterion	Marks
Identifies and explains three relevant code concerns	3
Implements the required improvements correctly	3
Demonstrates and records the five required checks	2
Provides constructive peer feedback and responds appropriately	2
Total	10